John Knox

RaceTrace: A Real-Time Group Tracking App

Computer Science Tripos, Part II

Churchill College

May 16, 2013

Proforma

Name:	**John Knox**
College:	**Churchill College**
Project Title:	**RaceTrace: A Real-Time Group Tracking App**[1]
Examination:	**Computer Science Tripos, Part II (2013)**
Word Count:	**11864**[2]
Project Originator:	Dr John Fawcett
Supervisor:	Dr John Fawcett

Original Aims of the Project

To create an Android application that allows multiple participants of some track based activity to continually view each others progress. The display would be accurate enough to provide useful location information in such a situation. The communication was to be implemented in more than one way, so that an investigation could, and would be done into the relative effectiveness of at least two different protocols.

Work Completed

I have created an Android application that achieves this by first letting users connect to each other in one of two different ways, and then displaying a diagram showing the relative positions and trails of all participants on the screen. This diagram is continually updated with the latest known location of each user.

[1] Formerly titled "Socially Enhancing Track Based Sports", as in the project proposal.
[2] This word count was computed by TeXCount.

The inter-device communication has also been implemented in two distinct ways, the first using a client-server model, and the second being peer-to-peer with various further protocol options for each model. The networking technologies used are Wi-Fi and cellular data networks.

I have also conducted a qualitative and quantitative evaluation of the system, including consideration for the security issues involved. The work completed fulfils the project requirements.

Special Difficulties

None.

Declaration

I, John Knox of Churchill College, being a candidate for Part II of the Computer Science Tripos, hereby declare that this dissertation and the work described in it are my own work, unaided except as may be specified below, and that the dissertation does not contain material that has already been used to any substantial extent for a comparable purpose.

I give permission for my dissertation to be made available in the archive area of the Laboratory's website.

Signed

Date

Contents

1 Introduction **1**
1.1 Context and Motivation 1
1.2 Initial Direction 2
1.3 Scope of the Work 2
1.3.1 User Interface 3
1.3.2 Session Setup 3
1.3.3 Location Sensing 3
1.3.4 Inter-Device Communication 3
1.4 Problem Description 3

2 Preparation **5**
2.1 Resources Available 5
2.1.1 Networking 5
2.1.2 Processor 6
2.1.3 Memory 6
2.2 Networking 7
2.2.1 Reliability of Cellular Internet 7
2.2.2 Unsuitability of TCP 8
2.2.3 Client-Server vs Peer-to-Peer Systems 8
2.2.4 Network Address Translation 9
2.3 Location Data 11
2.3.1 Obtaining a Location 11
2.3.2 Displaying Locations 11
2.4 Review 13

3 Implementation **15**
3.1 Software Structure 15
3.2 Software Engineering approach 16
3.3 User Interface 17
3.4 Session Representation 19

3.5 Session Setup 20
3.5.1 Bluetooth Session Setup 21
3.5.2 Remote Server based Session Setup 21
3.6 Map Display and Data Structures 23
3.6.1 Storing Location History Data 24
3.6.2 Drawing to the Screen 25
3.7 Data Sharing 27
3.7.1 Network Stack 28
3.7.2 Asynchronous Update Notifications 29
3.7.3 Iterative Construction of a Simple Server 29
3.7.4 Message Types 29
3.7.5 Peer-to-peer Version 30
3.7.6 Requests 30
3.7.7 Network Layer Resilience 31
3.8 Instrumentation 32
3.9 Concurrency 32
3.10 Review 33

4 Evaluation **35**
4.1 Run-Time Performance of Session Setup Procedures 35
4.2 Security and Privacy Issues 36
4.3 Comparison of Network Architectures 38
4.4 How the Application Scales 40
4.5 Prioritising Data 41
4.6 Review 42
4.7 Meeting the Original Goals 43

5 Conclusion **45**
5.1 Future Work 46

Bibliography **47**

A Project Proposal **49**

B UML Class Diagram **59**

C Structuring the Device Space **61**

D Example Log File **63**

E Format of Messages **65**

F PathCache Source Code **67**

Acknowledgements

I would like to thank Dr John Fawcett for providing the original idea and his valuable guidance throughout the year.

Chapter 1

Introduction

1.1 Context and Motivation

With the sudden rise in popularity of smart phones in the past decade, new innovations are being discovered at a rapid pace. One niche within the applications market is the area of physical sports enhancement. There are now multiple widely used applications[1] on the market offering real-time as well as historic personal tracking and analysis for the duration of some physical activity, be it cycling, running, skiing or something else altogether. Typically these applications display the location of the user on screen as they advance around some course, and provide statistics along the way such as velocity, distance travelled and many others. They also allow the user to store records of their session to compare with future performances and do further analysis. However, despite the many features on offer, none of these solutions acknowledge the fact that often the activities involved are group events, thus failing to capitalise on an entire dimension of available data: the performance of all of the other participants with whom the user may be competing with[2].

In the video games industry it has long been standard practice for racing games to display a simplified map of the course on the screen (typically in one of the corners) showing the location of each competitor. This allows the user to continually evaluate their relative position and stay informed throughout the duration of the race. While there are arguments for and against such a feature[3],

[1] Two popular examples at the time of writing are Endomondo and Strava.

[2] The social networking "share" paradigm has been applied to such applications, allowing others to see user statistics after the event has taken place but, to my knowledge, there is no real-time sharing facility allowing others to compete at the same time.

[3] Knowing the location of other participants can increase competitiveness and therefore improve performance, but it can also result in degraded performance due to winners complacency

its unanimous inclusion in gaming is sufficient proof that it is, at the very least, sometimes desirable.

1.2 Initial Direction

Taking a step back for a moment and considering the alternatives to developing a smart phone application, there are a variety of ways such a system could be implemented. One approach to solving this problem would be to develop an application specific device for the job. If this approach was taken then a multitude of options would open up as to what hardware should be used. The networking aspect of the device would perhaps be the most interesting with the large variance between available options.

One reasonable option would be to use a private radio system. In a typical use case of the desired system, the distance between participants would not exceed the range of available portable license-exempt radio modules. This means the user could support their own network infrastructure without relying on external providers or being subject to external costs.

While designing a custom device would have some definite advantages, with the widespread adoption of smartphones - most of which have all of the necessary hardware for this task, there really is no need. In order to achieve widespread adoption, the transaction cost should be minimised, so requiring additional hardware would be an unnecessary disadvantage when compared with competing applications.

On top of that, current high end mobile phones have capabilities far exceeding the requirements of such a project, making the design process easier and allowing for more complicated features to be added to the basic concept. For these reasons I opted to create a smartphone application. At the time of the decision, there were three main mobile application platforms, namely Apple iOS, Google Android, and Microsoft Windows Phone 7. The Android platform was chosen out of familiarity with the Java platform as well as having multiple devices at my disposal.

1.3 Scope of the Work

In order to achieve the goals laid out at the start four main features are required. These are outlined below.

---

and losers frustration.

1.3.1 User Interface

The application requires a graphical interface for displaying the position of the other participants in relation to the user. It was not my intention to produce a production grade user interface though I recognise that this would be a useful addition to the project. The only requirement was that it be effective and reliable, this has been achieved.

1.3.2 Session Setup

In order for inter-device communication, there needs to be a method of initial synchronisation so that each device knows (among other things) where to send data. There are various possible approaches to this problem. I have implemented two alternate methods, each with its own merits.

1.3.3 Location Sensing

Clearly some method of determining the current location is required if this information is to be shared with other devices. This was achieved using a combination of data from the Global Positioning System (GPS) and the cellular network.

1.3.4 Inter-Device Communication

The way devices communicate with each other after session setup is crucial to the project. The Internet API provided by the Android operating system (OS) was used for all such communication.

1.4 Problem Description

Participants do not always have up to date knowledge of the positioning (past and present) of all other participants. I solve this problem for the cases where all participants have Internet connected Android devices.

Chapter 2

Preparation

In this chapter I outline the theory behind the problem, and the work conducted prior to development.

2.1 Resources Available

2.1.1 Networking

Inter-device communication is vital to this project. Fortunately, there are multiple methods of communication on offer to mobile developers. Practically all mobile phones are now connected to the Internet, many with high speed access thanks to 3G and 4G technologies, as well as having 802.11 WLAN and Bluetooth.

Communication can be split into ad-hoc and infrastructure methods. Bluetooth falls under the Ad-hoc category, where there is no external networking infrastructure, along with Wi-Fi direct, a protocol available in the latest Android versions allowing Wi-Fi to be used in Ad-hoc mode without the need for an access point.[1] Wi-Fi is more commonly used in infrastructure mode, as an alternative to the cellular network for Internet access. The Android OS also allows devices to share an Internet connection via Wi-Fi or Bluetooth.[2]

On the Android devices available during development, Wi-Fi direct was not available since this is exclusive to later versions of Android. The cellular data connection was the chosen method of communication since it provides uniform access between pairs of devices whenever they have signal. This is compared to ad-hoc methods such as Bluetooth where opportunistic transfer would have to be relied on and application performance would vary widely depending on the differing spatial distribution of participants. For example a chain of participants where each is within range of the next would allow for a fully connected network,

whereas any other arrangement would not, resulting in partial data showing on screen until the networks reconnect. In the worst case if no participants were within ad-hoc range of any other for the entire duration of the session then no data would be transferred rendering the application useless.

Adding opportunistic ad-hoc communication would bring advantages, and it is elaborated in Section 5.1, but it would considerably increase the scope of the project so it has not been explored further.

2.1.2 Processor

The processing power of available Android devices varies widely, from top-end models having quad-core processors and dedicated graphics, to budget devices performing several orders of magnitude worse on CPU benchmarks.[3] Fortunately even the low end devices are sufficient for the computation required for this task. These requirements are[1]:

- Send and receive multiple network packets per second.
- Encrypt and decrypt multiple packets per second using some encryption scheme.
- Scan through arrays spanning hundreds of data points, drawing each to the screen without delay to the user.

2.1.3 Memory

In order to function properly, each device must record a trace of its and the other devices' history. This has the potential to be a large amount of data so the requirement was estimated before proceeding with the project. A typical use scenario would not normally exceed 4 hours. Say each data point is represented by 12 bytes: 3 32-bit integers[2] storing an x-coordinate, a y-coordinate, and a time value. At one update per second per device the required memory to store the history would be:

$$size = 1 \times 60 \times 60 \times 4 \times 12B \tag{2.1}$$

$$= 172,800B$$

[1] These requirements are purposefully not quantitative. This is because to an extent the system could be designed down to the capabilities of the device, while still offering value.

[2] 32-bit integers are likely excessive since relative positioning could be used with 1 metre (or variable) granularity. Depending on the scale, an 8-bit char may suffice, but at this stage, a cautious estimate was desired.

That is, < 169KB for each device. Clearly the requirement will scale linearly with the number of devices.

All Android devices provide each application with at least 16MB of allocatable memory. Bearing this in mind, the limit on number of users (for such a long session) will lie somewhere between 50 and 100, far higher than in a typical use case.

2.2 Networking

2.2.1 Reliability of Cellular Internet

The portability of mobile phones brings some disadvantages. One being that signal strength varies widely as you travel, depending on proximity of base stations and characteristics of the surrounding landscape.

Typical uses of sports tracking applications include mountain biking and skiing, activities such as these raise concerns due to the nature of their location, often being within dense areas of forest and in remote areas that wouldn't usually be prioritised for by network providers. For these reasons, signal strength in these locations is typically weak, often cutting out entirely and though significant effort has been made to avoid packet loss[3], during disconnection it is still a problem since different IP addresses are usually allocated on reconnection.

This means an application relying on cellular coverage for communication would need some resilience to disconnection, and should use the available throughput efficiently so that important data is prioritised.

Differing Network Characteristics

During some preparatory tests it was found that the characteristics of the two available 3G networks differed somewhat. While both the Three and Vodafone networks had similar bandwidth and latency, probing the networks showed that packets quickly queue up on the Vodafone network, perhaps as a penalty, whenever many packets are sent in quick succession. This results in effectively huge reception delays (up to 2 minutes). The way to avoid this is to bucket the data into larger packets to reduce the routing load on the network. On the Three network, no such throttling was apparent.

[3]Extensive link-layer retransmission protocols have been developed to minimise wireless packet loss, though such retransmission is hidden to the over-arching transport protocol which only sees high levels of jitter.[4]

This meant that any protocols developed should attempt to minimise unnecessary network routing, and perhaps collect statistics to infer such network characteristics and behave accordingly.

2.2.2 Unsuitability of TCP

The Transmission Control Protocol (TCP), uses a congestion control system that interprets all packet loss as the result of network congestion. When losses are detected, it reduces the transmission rate in the hope of reducing the level of congestion.

Given that this application will be subject to (possibly heavy) packet loss due to wireless disconnection and interference issues, TCP would not be an appropriate protocol to use. This is because TCP would interpret the packet loss as due to congestion and back off unnecessarily, resulting in much lower throughput than what is possible. Because of this, UDP was used instead which employs no congestion control, allowing arbitrary transport protocol features to be built on top.

2.2.3 Client-Server vs Peer-to-Peer Systems

Given that the cellular data connection is to be used for all communication, there are two ways the system could be realised.

One approach uses the support of remote servers to act as intermediaries between clients. The other approach is to use only client systems with no external processing other than the existing network infrastructure.

The main advantage to the peer-to-peer approach is scalability. Peer-to-peer systems are self-sufficient and would allow unbridled scale if the application was to be widely used.

The client-server system on the other hand would require servers to be maintained at the cost of some party. If the application vendor provided this service then more resources would have to be provided as the number of users increases. As well as this it would introduce a single point of failure which does not exist in the peer-to-peer model.

There are advantages to the client-server model however, for one it avoids the NAT traversal problem outlined in Section 2.2.4.

Another more subtle advantage of using a server is an improvement in transmission success rate. This may seem counter-intuitive since an extra hop is required for each packet[4], but the logic is explained below.

[4] "Hop" here is not used in the literal network routing sense, but rather refers to the path

In the event of high packet loss (considered likely given the application), packets can be lost when transmitted from the phone to the base station and vice-versa. For peer-to-peer and client-server systems alike, the chance of a successful exchange (wireless transmission and reception) with one attempt at sending is:

$$P_{success} = P_{transmit} \times P_{receive} \tag{2.2}$$

and assuming[5] $P_{transmit} = P_{receive}$ we have:

$$P_{success} = P \times P \tag{2.3}$$

However when repeat attempts are allowed from the intermediate server, the probability of success lies in favour of the client-server system:

$$P_{success} = P \times (P + \gamma) \tag{2.4}$$

where

$$\gamma = (1 - P)P + (1 - P)^2 P + (1 - P)^3 P + ... \geq 0 \tag{2.5}$$

and since

$$P + \gamma = \sum_{i=0}^{\infty} (1 - P)^i = 1 \tag{2.6}$$

we have:

$$P_{success} = P \tag{2.7}$$

The reason for the increase in probability of success, is that packets reaching the server but not the end client are not completely lost; they may be retransmitted from the "halfway" point. Therefore even when repeat transmissions are enabled from the initial sender, the client-server model will always have a probabilistic advantage where there is packet loss[6].

2.2.4 Network Address Translation

With most Internet services still using Internet Protocol Version 4 (IPv4), addresses now come at a premium with many ISPs using Network Address Translation (NAT) to expand their internal network address space. With the transient

increase from [A] to [B] in peer-to-peer communication, to [A] to [Server] to [B] in Client-Server communication.

[5]This assumption is not strictly necessary but done here to simplify the resulting expressions.

[6]Assuming that packet loss between the base station and server is negligible compared to loss between the client and base station, and that not all dropped packets are retransmitted at the Data-Link layer.

nature of mobile phone data connections, most if not all mobile ISPs fall into this category. One problem with this technology is that it can make peer-to-peer connections difficult or even impossible, as explained below.

NAT occurs at the router between an internal network and an external network. The router allocates internal addresses to the internal nodes, but only the router has an external network address. When an internal node sends a packet to some external node, the router translates the "from address" to its own external address so that the receiver may reply. The router can then distribute externally arriving packets to internal nodes based on the TCP/UDP port numbers and sender address using the state it set up earlier when routing outgoing packets.

The result is that many internal nodes can share a single external IP address, sidestepping the limited address space problem.

Problems arise when you try to connect to a node behind a NAT. If this is the first point of contact, then the router has no relevant state to determine which internal node to forward to, rendering it unconnectable unless it initiates the connection itself. With both clients behind different NATs, then neither one can connect to the other. UDP hole punching is a technique used to overcome this by getting both clients to initiate the flow using the same port numbers. Both NAT routers each see the outgoing packets and set up state so that the incoming packets will be directed to the right node.

Tests were conducted to find out whether peer-to-peer communication would be possible over cellular data connections. During these tests, access to two 3G data networks, Three and Vodafone, was available. To mitigate risk these tests were carried out early in the project in case the plan had to be changed as a result.

Using UDP hole punching external connections were made to an Android device over the 3G network provided by Three, which would ordinarily provide the means to achieve peer-to-peer communication. However in this case, there were further complications meaning that despite packets being successfully sent from external networks, packets from within the Three network to other addresses also within it were unsuccessful. It was concluded that Three does not allow intra-network communication meaning true peer-to-peer protocols are not possible when limited to this network provider.

Since external connections were successfully achieved, it is assumed that peer-to-peer communications would be possible when a mix of network providers is used.

Unlike Three, in the experiments conducted Vodafone always remapped the port numbers of outgoing messages, stopping UDP hole punching from working, thus disallowing peer-to-peer traffic on the Vodafone network.

In light of these discoveries, it was clear that a peer-to-peer system would not be universally accessible, but may work in theory when certain networks are used, for example between two separate networks operating using the system that Three employs.

The transition from IPv4 to IPv6 is due to massively increase the Internet address space, which should eliminate the need for NAT, making peer-to-peer mobile applications more practical and universally accessible.[5]

Therefore, an implementation of the system using peer-to-peer protocols would still have benefits (even if not immediately), so the plan to make both a client-server implementation and a peer-to-peer one was continued.

2.3 Location Data

2.3.1 Obtaining a Location

Most Android phones are able to use data from both GPS satellites, the cellular network and Wi-Fi if connected to identify their location. The Android OS provides location estimates from each source when available, along with an estimate of their current accuracy. This allows applications to combine different location providers to use the most accurate one at the time. It also provides a measure of redundancy meaning long intervals without location updates are rare. The power consumption of the device increases as more location providers are used, with GPS being the most power hungry. For this reason, update intervals can be specified so that the providers will only update their location value at specific time intervals.[8] Since this particular application is aiming for real time updates and bearing in mind its competitive purpose, relatively frequent updates are desired. In all tests conducted, the frequency of updates used was one second and it was observed that on the test devices available, battery consumption was very reasonable.

2.3.2 Displaying Locations

When drawing the trail of a user to the screen, it is important for the image to appear in agreement with the path the user has experienced. Since we, as humans, don't notice the curvature of the earth in everyday activities, we perceive it as flat. So when we walk the perimeter of what we think of as a square on the surface of the earth, we expect to see the drawn path appear as a square, when in reality it is a non-planar 3-dimensional path.

Android provides location data in the form of latitude and longitude values. Because the form of the earth is closer to that of an ellipsoid than a sphere, simply plotting the latitude and longitude as y and x coordinates on the screen would cause the image to appear skewed. To account for this, a coordinate transform must be applied to map this data onto a suitable geographic coordinate system. There are four main systems in use: Latitude and Longitude (LL), Universal Transverse Mercator (UTM), Universal Polar Stereographic (UPS), and Ordinance Survey Great Britain (OSGB). UPS is a system designed to cover the polar regions, which are not covered by UTM. This option was discarded because for this project, operating in the polar regions is not a concern. This leaves UTM and OSGB as possible choices.

Both transforms are based on an ellipsoidal model of the earth, though UTM uses the more universal WGS84 model. Being a transverse Mercator projection, UTM has the property of conformality, meaning that it preserves angles but distorts distance and area (over large distances across the earth), whereas OSGB is a Mercator projection meaning it is not conformal so does preserve distance and area but not angles.[6] It also only covers the area in and around Great Britain.

In order to transform coordinates from LL to one of these systems, there is no single map transformation to apply. For example, in UTM the surface of the planet is divided into 60 geographical zones based on longitude and a different transverse Mercator projection is used for each zone. This means calculation is rather involved but fortunately there are various open source libraries available that offer conversion between the systems. Of these JCoord was chosen because out of those available in Java with suitable licensing, it was the most lightweight and provides both OSGB and UTM coordinate systems.[7]

To decide which projection (if any), should be used, the map drawing function of the application (described in Section 3.6.2) was used to trace the path of a user when walking the outline of certain shapes. The outline (including the boxes) of a football pitch was walked while using the application, and the shapes drawn using different coordinate systems were compared to detect any severe distance scaling using the relative sizes of the boxes, or any angle distortion from the right angled corners of the pitch. Using a sports pitch also has the advantage that the surface is relatively level and flat.

When drawing the raw longitude and latitude as x and y coordinates, there was clear distortion of angles, causing the pitch to resemble a parallelogram rather than a rectangle. This deformity was also observed, to a lesser extent when using the OSGB transform. However, when using the UTM transform, there was a large improvement in the shape of the trace. Upon careful inspection the edges were not perfectly perpendicular, but this error was not easily noticeable and certainly

good enough for the desired application. No distance or area deformities were detected.

With these results, the UTM coordinate system was chosen since the resultant shapes represented the pitch most accurately[7].

2.4 Review

The preparation stage went according to the proposed plan. In light of these discoveries, the original time-scale was still considered reasonable and was not altered.

[7]The pitches used were located at Churchill College Grounds, Cambridge.

Chapter 3

Implementation

This chapter explains how the software was built and how the major components were implemented.

3.1 Software Structure

The client software is composed of the following 8 packages:

graphics: Responsible for graphical manipulation other than the simple menus provided by the Android SDK.

gui: Responsible for the menu screens.

location: Responsible for obtaining location data.

network: Responsible for network communication.

protocol: Responsible for protocol implementation and coordination.

session: Responsible for session setup and persistence.

settings: Contains the configuration file and miscellaneous enumerations.

store: Responsible for storing location data, updating it and notifying other system components of updates.

A simplified dataflow diagram of the software is presented in Figure 3.1. The Setup activity guides the user through the session setup process, interacting with the Session Setup class which does the work. This yields a Session object which is given to the Protocol Manager class and used to initialise the protocol. At the same time, control is passed from the Setup activity to the Map Display

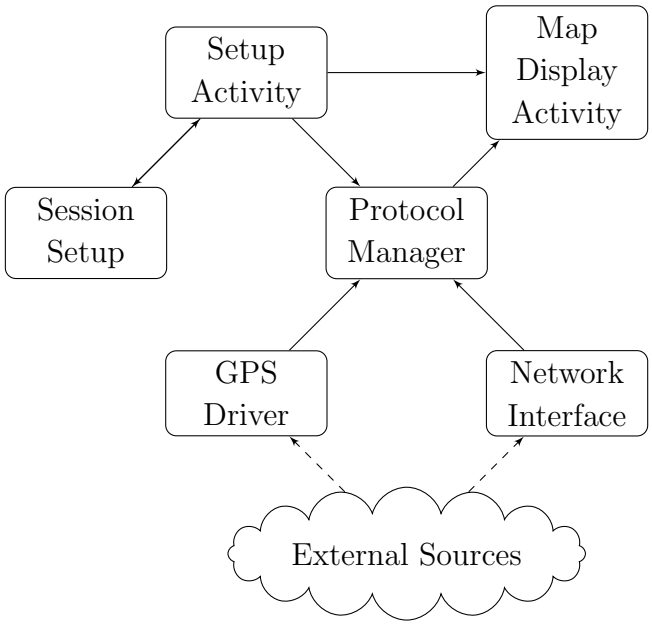


Figure 3.1: Diagram showing the flow of data between core system modules.

activity, which gets asynchronously updated as a result of the work of the Protocol Manager, whose role is to coordinate the incoming and outgoing data.

A Configuration class was used to store all numeric values and enumerations within the system making it completely tunable, with a single point at which all changes can be made.

3.2 Software Engineering approach

Throughout the project, all code was stored on, and regularly pushed to a remote Git repository on the PWF space to provide backup and version control. The entire project was also continuously backed up using Dropbox.

An iterative development process was used for the main construction of the application which is described below. Fresh Git branches were created and merged for each iteration.

In Android development, decoupling the User Interface (UI) from the back end computation is simple: each "screen" has its own activity class which interacts with the system components through their public interfaces. This allowed the UI to be built without yet having any "behind the scenes" code.

The UI was built first (including the map display screen) to allow for testing of the other components as soon as they were ready.

With the bare UI in place, the core framework of the system was built next. This was a collection of classes that would hold the session state throughout the use of the application, and which the public methods of the various isolated components of the system would be called from. The core includes classes representing Devices, Sessions, Protocols etc. Gaps with strictly defined interfaces that would be implemented by specific modules were left at this stage. This was aided largely by the following of Object-Oriented Programming (OOP) principles which were used throughout the entire project. As an example, Information Hiding is the process of defining standard interfaces for components and hiding the implementation so that internal changes may be made without affecting other components. This proved invaluable when multiple alternative modules were implemented, such as the different session setup procedures.

Much care was taken at the start of the project to define the classes and structure of the software before construction began, so that each package and component would have a specific purpose (tight cohesion), and that they would be loosely coupled, meaning the points of interaction between them were minimal and well specified. To do this a UML class diagram was drawn up to provide a reference point for the interfaces and overall application structure. This was very helpful during the implementation stage since all structural and interface decisions were documented here as they were made, and the diagram was regularly updated as the project advanced. The diagram is presented in Appendix B.

With the core structure in place, the next step was to iteratively add components to the system until it was fully functional, and then to continue iterating, adding further improvements.

Progress was tracked using the fortnightly goals outlined in the proposal. This was fine-grained enough to show me when I was on track to within a week, so I was able to complete all the "additional" tasks given, without worrying about time constraints.

3.3 User Interface

Although this project was not concerned with the interface ease of use, some graphical interface was necessary, and to best represent the systems capabilities a reasonable design was attempted. A cognitive walkthrough was done to identify possible user aims and common decisions that would have to be made, and the "menu screens" were wireframed by hand to quickly prototype a design.

In the Android ecosystem, user interface screens are represented by Activities. Activity is a subclass of the Java Object class, allowing arbitrary functionality

on top of the standard Android framework. Ordinarily each activity corresponds to a view of the screen and has its own layout, which is usually specified in an XML file.

The menu structure of the application is illustrated in Figure 3.2. Each of the three activities accessible from the main menu is a subclass of `SessionSetupActivity` and can be used to initialise a session.

When a session ends and participants exit the Map Display Screen, they return to the session setup activity. One thing the cognitive walkthrough identified was that immediately after one session, users are unlikely to want to start another one, so instead of prompting the user to setup another session, the `SessionSetupActivity`s callback method is overridden to take them back to the main menu.

As well as those in the diagram, there is a Preferences activity, which allows the user to modify the system settings.

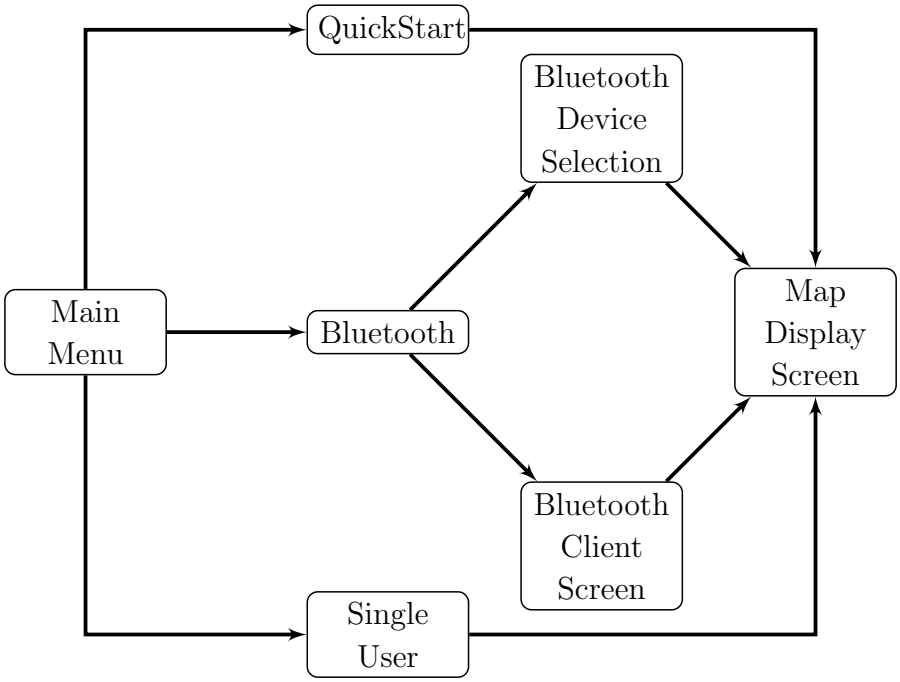


Figure 3.2: Flowchart showing the possible paths through the activities of the application.

The Android Software Development Kit (SDK) offers assistance in creating graphical user interfaces, including a "What you see is what you get" (WYSIWYG) graphical editor. However, for the purposes of this project this tool was unnecessary and it was decided that all GUI work would be done precisely by text entry and using relative positioning so no undesired artefacts would be ac-

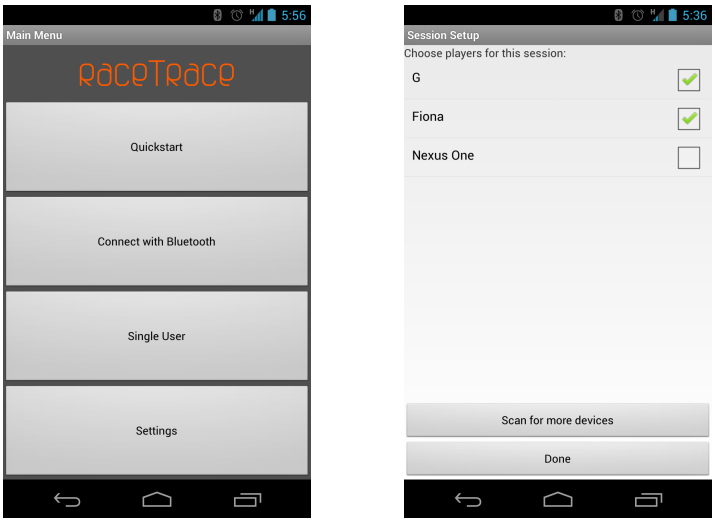


Figure 3.3: Screen shots of the Main Menu and Bluetooth User Selection screens.

cidentally introduced by the automated tool. Such artefacts could be hidden during tests depending on screen size and resolution and lead to troublesome bugs. However the tool was useful for visualising changes made to XML layout files without having to re-install the application each time.

3.4 Session Representation

As far as this project is concerned, a *session* is the recreational activity taking place from when the participants start to move until they all cease to move and the physical activity is considered finished.

A Session object, in this case, is a digital representation of such a *session*. It is implemented as a Java class containing both an ordered list of Device objects and a set of cryptographic keys for communication if encryption is used. The Session class follows the Singleton design pattern, ensuring that no more than one instance of it exists at any one time.

A Device object consists of a name, a DeviceHistory object which is where the historic location data is kept, a DeviceHandle which is an instantiation of a subclass of the Absract class DeviceHandle containing the address information for this Device, and a DevicePath which is used for drawing and is explained in Section 3.6. In this project, all communication was via the Internet, so DeviceHandleIP is the only implemented subclass of DeviceHandle, other than DeviceHandleSingleUser which is used when there is only one participant.

3.5 Session Setup

The application structure first implemented left a gap for a session setup module. The reason for this is that there are numerous ways to setup the session and it was conceivable that there may not be an absolutely optimal method.

As justification, consider the case of using the client-server model for main inter-device communications. With this topology in place, the server can act as an intermediary when setting up the initial session, since its address is always known by all participants. However, if peer to peer methods alone are used for communication, then there is no oracle that can introduce devices to each other, and some ad-hoc discovery method would be required.

At the initial stages of development, a simple single-user module was used to initialise the session, allowing testing of the data storage and visualisation components before further setup modules were implemented. As well as acting as a temporary placeholder, this module is activated in the final system whenever the user selects the "Single User" option at the main menu.

The first multi-participant session setup module was designed to be universal, since at the time developing further modules was an optional project extension so this may have been the only one. This meant that ad-hoc discovery methods had to be used since they don't rely on an external provider and can work with both server based and peer to peer systems.

There are several options for such ad-hoc device discovery.

All devices running Android version 2.2 and above that are Wi-Fi enabled are able to host personal Wi-Fi hotspots. One solution to the setup problem would be to have one device set up a hotspot and the other participants connect to that hotspot. The host could then identify and add all connecting devices to the session, and distribute the session information back to them over the connection.

A similar suggestion is to use Bluetooth as the medium. This would operate in a similar manner to using Wi-Fi.

Near Field Communication (NFC) has recently been adopted by handset manufacturers, and can initiate communication when two NFC-enabled devices come into contact.[8] QR codes allow communication via a visual channel and could be used by displaying a QR code on one screen while another devices directs their camera towards it. These two methods are similar in that they provide one-to-one communication and require one device to be physically oriented in respect to the other. This means coordinating a large group of devices could become a complicated process for the user, even with on screen instruction.

Bluetooth was chosen over Wi-Fi because it is a completely ad-hoc form of communication while using Wi-Fi hotspots is not. Also for all users to connect to

the hotspot the host would have to reveal their personal hotspot password, unless it is not encrypted, both situations are far from ideal. Wi-Fi direct would have provided a superior option but is not yet supported by most devices (including those available for this project).[1]

3.5.1 Bluetooth Session Setup

In order to initialise and distribute the session using Bluetooth, the following steps are taken:

- The (user elected) master user selects the desired Bluetooth devices from a list on screen as shown in Figure 3.3.
- All other (slave) users open a common (known) port and start listening for a Bluetooth connection.
- The master device connects to each selected device in turn, retrieving its address information, as well as any necessary cryptographic information such as public keys.
- The master device marshals a session object from all of the collected information (including its own).
- The master device connects again to each slave device in turn and sends them the serialised session object.

The available devices are retrieved from the OS using `getBondedDevices()` and the names from these are presented in the form of a ListView. ListView is provided in the Android API and provides a way to display lists of items on screen within an activity layout. When the choice mode is set to "multipleChoice" it lets the user select multiple items and provides the results in the form of a boolean array. This lets the user select which of its paired Bluetooth devices it wishes to participate with.

The master then goes through this list one by one and connects to the Bluetooth ServerSocket that each slave device has opened, and uses these connections to exchange the required information.

3.5.2 Remote Server based Session Setup

While the Bluetooth based method worked well in testing, having to select multiple devices and then connect to them sequentially causes the process to take longer and require more effort than strictly necessary.

For this reason, another session setup procedure was built, using a remote server.

To connect, the user of each device presses the "QuickStart" button. This is the only user input necessary for QuickStart setup making it more convenient to use than the Bluetooth setup procedure[1].

When the function is activated the location of the device, obtained by the GPS Driver, is sent to the server along with the session information of the device.

At the server, these requests are received and grouped by location into session groups. From these groups, session objects are marshalled and sent back to the devices involved.

The location information used for grouping comes in the form of 2D Cartesian coordinates.

```
for device a: requests do
    a.wait();
    for device b: requests do
        if distance(a, b) ≤ x then
            a.add(b);
            requests.remove(b);
        end
    end
end
```

Algorithm 1: Naive grouping algorithm.

```
for device a: requests do
    a.wait();
    for Block block: a.getBlock() + b.getNeighbours() do
        for Device d: block do
            if distance(a, d) ≤ x then
                a.add(d);
                requests.remove(d);
                block.remove(d);
            end
        end
    end
end
```

Algorithm 2: Improved grouping algorithm.

[1] If Internet access is available.

If a naive grouping algorithm such as Algorithm 1 was used the run time complexity would be $O(n^2)$ where n is the global number of devices currently starting sessions. This is avoided by noting that there is an upper limit on the distance between any two devices considered "close" to each other. This allows one to segment the 2D location space into a grid of disjoint areas, where comparisons need to be made only between points in adjacent blocks, as is done in Algorithm 2. Note that both Algorithms 1 and 2 require the list `requests` to be sorted in order of request time. `requests` is a FIFO queue so this happens naturally as the list gets populated. `a.wait()` simply sleeps until request `a` is old enough to be grouped. This waiting ensures that nearby devices have ample time to connect to the server without missing the opportunity. While still technically remaining $O(n^2)$ in the worst case (where all devices are close together), in practice there is an upper limit on the number of users that can be in the same place at the same time resulting in an effective O(n) runtime complexity to process all groups.

In practice when this method is available the speedup compared to the Bluetooth method depends on the length of time that the server waits before grouping requests. One important benefit is that it scales gracefully with multiple devices unlike the Bluetooth method which is serialised. While being more convenient this comes at the cost of expressiveness since arbitrary participant selection is no longer possible. If desired, an option could be added however to allow selection of participants without much further work.

The QuickStart setup procedure can be used to initiate peer-to-peer sessions, not just client-server ones, combining ease of use with partial resource distribution—while a server is required, it only has to group requests and not redirect all traffic, so would be more efficient to run than a complete server.

Clearly the Bluetooth method has the important advantage of not relying on an external provider.

3.6 Map Display and Data Structures

There are several ways the location data could be visualised on screen. The approach taken here is to draw a line to the screen for each user in the session, showing their entire path since the beginning of the event. The lines could be superimposed onto existing map visualisation data such as that provided by Google[2] but with the desire to keep the interface simple and quickly interpretable this wasn't attempted. An example of the display output is given in Figure 3.4.

[2]Map data available at https://developers.google.com/maps/documentation/android/

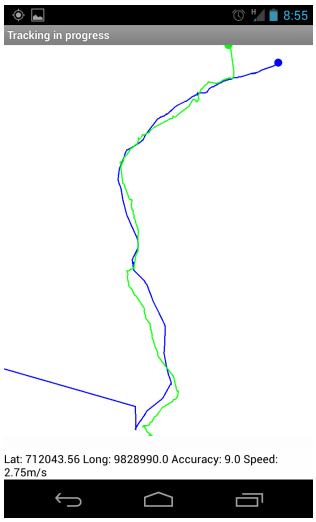


Figure 3.4: Screen shot of the Map Display Screen showing the visualisation of data during a field test.

The data displayed here is from a genuine field trial. Notice that the intricate deviations of each user from the "beaten track" are evident. When the shot was taken, the two users had currently parted ways and would shortly rejoin the same route again. This is where the application displays its value, since without it each user can only speculate about the position of the other. Notice also the artefact at the beginning of the blue coloured trail. This was introduced at the start of the session when the position accuracy was low. When a better fix is provided the line appears to "jump" straight to the improved location.

3.6.1 Storing Location History Data

For each device, the entire known location history, whilst in the current session, should be stored at every other device so that each device can potentially perform arbitrary computation and aggregation on it.

To do this, a data structure was required that would store a list of (time, location) data points.

The structure chosen was a linked list of array lists. The data points are stored in fixed-size arrays, with new arrays being allocated when needed. This means append, lookup and insert have runtime costs of $O(\lceil N/L \rceil)$ where N = number of points and L = array size.

For each device, this structure is indexed by an integer logical clock governed by the device the data originates from. This means the time between location updates is varied depending on the speed of travel resulting in constant spatial

resolution. It is achieved by each device incrementing its logical clock whenever it travels some fixed size distance. It also provides a simple way to check for missing data, since if there exists any empty index i such that $i < j$ where j is the latest known index, then the data for index i is absent and should be requested. Requests are explained further in Section 3.7.6. One alternative would be to index the structure by time, which would then suffer from variable path resolution depending on speed, and several undesirable effects such as duplicate locations being stored whenever a device remains stationary.

Since the aim is to fully populate the list with data points up to some logical time l, allocating contiguous arrays is not wasteful because we always aim to fill all current gaps. The trade off here comes with setting the length of the array chunks—too large and more memory will be unused at the end of the array, too small and lookup times will increase because of the increased size of the linked list.

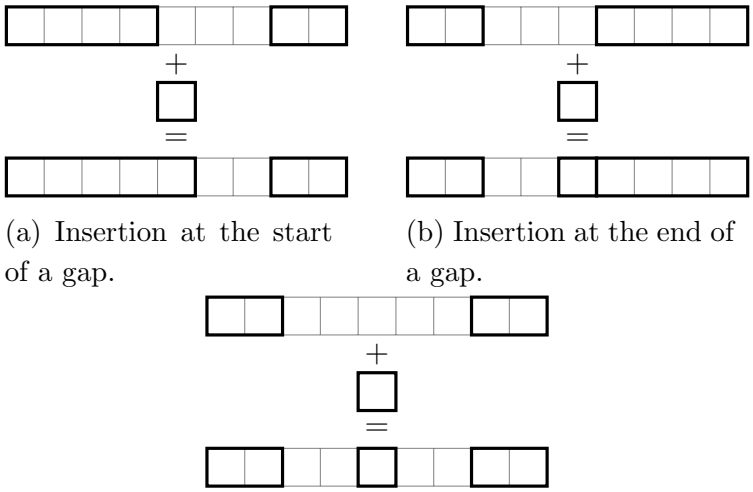
3.6.2 Drawing to the Screen

To present a visualisation to the user, the data points need to be mapped onto a 2D space the size of the device screen, and lines need to be drawn between them.

The Android canvas API provides suitable methods for plotting points, lines and paths to the screen.[8] In this context, a Path object represents an append-only ordered list of points that can be pre-computed and drawn to the screen efficiently at different times through the use of the canvas of a View object. Being append-only, the Path objects don't provide an immediate solution for drawing entire device histories to the screen for each frame. The reason for this is that at any point in time, there may be absent data points between two present data points. Since these absent points may later be obtained, the data structure should allow for insertion and not just append without having to reconstruct the entire Path object again.

A View object is a Java object that can be embedded into the layout of an activity. Whenever the activity is created, so are the embedded View objects. A custom view object was created, named a MapDrawer, which is a rectangular canvas that can be displayed on screen, and has access to the PathCache structure described below for drawing the traces.

Figure 3.4 shows the MapDisplayScreen activity, the main element of this screenshot which contains the different coloured lines is the MapDrawer Object.



(a) Insertion at the start of a gap.

(b) Insertion at the end of a gap.

(c) Insertion into the middle of a gap.

Figure 3.5: Figure showing how insertion into the PathCache occurs in different contexts. Boxes of indices denote Complete Segments, sets of consecutive unboxed indices denote Gap Segments.

The PathCache

To solve the Path insertion problem, an object named a PathCache was designed and implemented. This is comprised of a TreeSet of Segment objects, where a Segment may be either a Complete Segment or a Gap Segment. Complete segments represent parts of the data point list that have no missing data. Gap segments are segments of the list that are empty. Each Complete Segment object has a Path object which contains its data points and can be drawn to the screen when required.

This structure provides efficient insertion of data points into the lists as well as conserving the efficient append operation when using a single Path object. Figure 3.5 shows how data points are inserted into the PathCache. To append points, they are simply appended to the Path of the last Complete Segment (the last segment will never be a Gap Segment because gaps are defined as missing segments whose index is less than that of the largest index received).

When inserting (not appending) new points there are two possible cases:

In the first case (Figure 3.5a) the new point is at the start of a Gap Segment, in which case it gets appended to the preceding Complete Segment, and the start of the Gap Segment is moved along by one, or removed it it was of length 1.

In the second case (Figures 3.5b, 3.5c) the new point is not at the start of a Gap Segment (meaning it is somewhere else in some Gap Segment). In this case

the Gap Segment is shortened, a single point Complete Segment added for the new point, and if the point wasn't at the end of the Gap Segment, then an extra Gap Segment is added to fill in the rest of where the old Gap Segment was.

The TreeSet[3] provides mapping from indices to Segments, and fetches predecessor and successor Segments in O(log(N)) time.

The exact source code for insertion into the PathCache is given in Appendix F.

While not necessary for an understanding of the structure, it may be noted that Gap Segment objects need not contain any information. The start of any Segment object (Complete or Gap) is defined by the integer index of it in the TreeSet and likewise its end is defined as the integer index of its successor in the TreeSet (unless there is no successor - this is handled by keeping a single "largest index" integer in the PathCache). The only function of a Gap Segment is to mark the end of the previous Complete Segment in the TreeSet, so `null` would suffice instead, however for the sake of good OOP practice a single instance of Gap Segment exists and is referred to in the TreeSet several times[4].

To draw the paths to the screen each frame, every entry in the TreeSet is iterated through (in index order), with the Path object of every Complete Segment found being drawn to the screen. For each Gap Segment found, a straight line is drawn from the end of the previously drawn Path to the start of the next as a way of interpolating the path between known data points.

While scanning through the Path objects, the boundaries are calculated, these are the maximum and minimum values occurring on each axis. They are used to calculate the scale factor required to map the life-size model to the scale of the screen. A matrix is constructed from this factor and applied to transform each of the paths, then they are shifted to be aligned at (0,0). This results in the paths always being displayed at the largest possible scale without distortion or exceeding the bounds of the screen. Such transformations are done using the Android graphic methods so that any GPUs and efficient device-specific methods are used.

3.7 Data Sharing

As mentioned earlier the networking parts of the system were sufficiently loosely coupled to enable modular swapping of different protocols. This was achieved us-

[3]TreeSet is an implementation of a sorted set structure, where a tree structure is used to store values for efficient (O(log n)) lookup.

[4]This allows for "cleaner", more uniform segment checking using `instanceof(GapSegment)` rather than `== null` .

ing standard OOP conventions and the use of an abstract class, ProtocolManager, which was subclassed by the various different protocol implementations.

The approach taken to send data was initially a simple bucket and send protocol. This means that as new points arrive from the location aware part of the system, they are put into a bucket and sending is delayed until either the bucket is full or the first item in the bucket is older than some fixed amount of time.

This prevents abusing the network by sending many smaller than necessary packets, as was seen to be a problem in Section 2.2.1, while avoiding excessive sending delays.

There are two ways new data can enter a client system: from the location aware part of the application or from the network. In each case the Protocol Manager adds the new points to the PositionStore. If the new points are not from the network, it also adds them to the outgoing queue to be broadcast to other devices.

3.7.1 Network Stack

The Open Systems Interconnection (OSI) model (Figure 3.6) is a conceptual hierarchy of 7 logical layers which together make up a communications system. Conceptually, messages are passed from one layer to the next after each has processed them.

Application
Presentation
Session
Transport
Network
Data-Link
Physical

Figure 3.6: Diagram of the OSI model (or stack) showing all 7 layers.

This application includes communication features in the Application, Presentation, Session and Transport layers. Abstraction of layers was used during development, to minimise inter-layer cross-talk, simplifying the development process.

3.7.2 Asynchronous Update Notifications

An interface `PositionStoreSubscriber` is defined that lets objects subscribe to updates from the PositionStore. Whenever new points are added to the PositionStore, it alerts the subscribers of the update (by calling `notifyOfUpdate()`) so that they can be notified and process the new data asynchronously. This is an implementation of the Observer design pattern.

The objects that subscribe to updates are the MapDrawer in the client application, and in the server application where there is no MapDrawer component the server specific session representation, ServerSession, subscribes. The ServerSession objects are responsible for managing the server-specific state and dispatching messages to clients.

Note that in the server application, where there are multiple instances of ServerSession and PositionStore, each ServerSession only subscribes to the PositionStore relevant to its session.

3.7.3 Iterative Construction of a Simple Server

For the first implementation, the client-server model was chosen. This meant that all communications between devices would be via a central server, effectively forming a high-level star topology. This simplifies the client side code since clients only ever need to send and receive messages to and from a single address. First the sending only aspect of the client side code was written, and a prototype Java server application was made that would print details of received data so that the client code could be debugged.

After this, the core framework of the client side application was added to the server, including the PositionStore and message formatting code.

The server acts like a client, in that it has the same data structures for storage, receives packets containing data points and consequently updates its state to keep track.

Originally the server application was made to serve only one session at a time, where the number of devices it would wait for (to set up a new session) was given as an argument on the command line. In isolation the device grouping code (Algorithm 2 from Section 3.5.2) was written and unit tested before being added to provide multi-session support.

3.7.4 Message Types

Two message types have been defined for inter-device communication once the main protocol is under way. The first is the payload type which carries historic

location data about possibly several devices. The second is the request type which lists the data points that are absent from the PositionStore of the sender. These message formats are given in Appendix E.

3.7.5 Peer-to-peer Version

For the peer-to-peer implementation of the system, each device keeps a DeviceConnection object for each other device. The protocol as usual is flexible and left to be implemented by the ProtocolManager subclass. In this case what has been done (in `ProtocolManagerP2P`) is when a device generates new points, it sends out the payload message to all other devices, by iterating over the DeviceConnections. Other approaches were identified during the preparation stage, such as forming a ring network or pairing up devices based on spatial locality, but due to time constraints these were not implemented. They are listed in Appendix C.

In `ProtocolManagerP2P`, requests are always sent to the generators of the requested points and responses are sent to either the requester or all other devices, as specified in the configuration file.

3.7.6 Requests

Since the User Datagram Protocol (UDP), an unreliable transport protocol, is used to transfer history data between devices, some method of retransmission is required if a complete knowledge of path history is desired at each device.

A request system was implemented which allows each device to determine the data it is missing, and request it from the network by way of a request message.

This is done by maintaining a list of the currently absent data points. When a new point arrives, if it has a larger index than any present data point, all indices from the current latest one up to that index are added to the absent list. If the point is not the new latest point, it is removed from the absent list.

With this absent list, requests can be made to the other devices and/or the server for them. Because of the monotonic indexing system for each device and the way the absent list is maintained, the presence of an index in the absent list implies that it *is* present at at least one other device.

The way that requests are made and responded to has been purposefully made flexible. The implementation details are left to the subclass of ProtocolManager that is being used. For the peer-to-peer system a single request message is periodically broadcast containing the entire absent list, whenever data is missing.

Several response decision policies have been implemented which govern how devices respond to requests. An enumeration in the configuration file specifies which policy is used. The implemented options are:

- Always respond if this device has any of the requested data.
- Respond with probability[5] p, if this device has any of the requested data.
- Respond if this device has $> n\%$ of the requested data.
- Respond with probability p where $p =$ the fraction of requested data that this device has.
- Respond if any of the requested data was originally generated by this device.
- Respond if, of the requested data, this device generated the largest proportion of it.
- Never respond.

For the client-server system, devices only ever communicate directly with the server, so can only make requests to the server. When this occurs, since the server too stores history data, it looks up the requested data in its own PositionStore. The requested data that it has are termed the *hits* and the remaining requested indices are the *misses*. The server replies to the requester with the *hits* in the form of a payload message. It then groups the list of *misses* by the device that originally generated them, and to each device it issues a new request containing that devices group. There is no value in requesting the data from any device other than the generator as no other device can possibly have it since all messages must have gone via the server. Since requests only go to a single device, the client-server system uses the policy "Always Respond".

3.7.7 Network Layer Resilience

When temporarily disconnected from the network, devices usually reconnect with a new IP address. To cope with changing addresses, the receiver of any packet always updates their record of the senders address (and port number) to the one it was just sent from, using the identity information contained in the packet. Provided at least one device is connected at all times (and retains its address), all other address information will eventually be re-distributed naturally.

[5]To trigger an event with probability p where $0 < p < 1$, a (uniformly distributed between 0 and 1) random Double d, is generated, and the event is triggered if and only if $d < p$.

3.8 Instrumentation

At times, random data was generated and used in place of real location information. A screen shot of the application using such a method is shown in Figure 3.7. This allowed for testing the display before the location-aware code was built, but also made testing more convenient when using Android virtual devices.

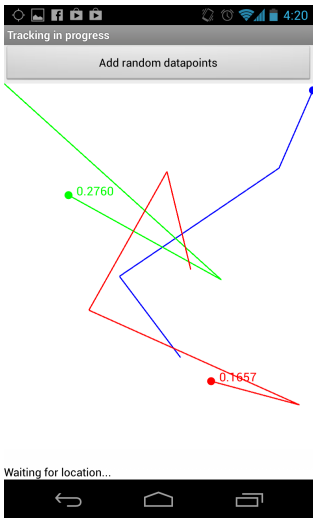


Figure 3.7: Screen shot of the application during a 3-device session using randomly generated location data.

At the start of the project, only one physical device was at hand for testing[6]. To test the message processing code at this stage, a "duplicate" option was added to the server causing it to duplicate every received point, change the device ID and translate the location by 10m. This extra point when inserted, simulated the interaction of a second device and would show up on screen as having a position right next to that of the real device.

A Logger class was also built to record of all notable events[7], this record is written to disk when the session ends, and was used to do a quantitative evaluation of the system in chapter 4.

3.9 Concurrency

One of the main challenges in the project was dealing with concurrent threads. This includes spawning threads when needed, synchronizing threads, ensuring

[6]Later, more devices became available.

[7]Events include receiving packets, issuing requests etc.

thread safety, and doing object de-allocation—a problem usually considered solved in sequential Java, but which brings new challenges when multiple threads have references to the same objects. In Android, each application runs as its own process, but freedom is given to use as many threads as desired. The development guidelines offer advice on using threads, an example being to never construct new data structures on the UI thread to prevent unresponsive delays in the user experience. All of the official guidelines were followed throughout development, and there were many points in the project where concurrency was used[8].[8]

Whenever concurrency was present, use of data structures and state was carefully tracked and the `synchronized` keyword was used to enforce mutual exclusion between possibly conflicting methods.

In addition to this, `destroy()` methods were defined for all classes with static state and these were called in a cascading fashion whenever de-allocation was due, such as the user exiting an ongoing session back to the main menu. Also a global synchronized boolean variable `alive` was defined in `ProtocolManager` which is checked for in all situations where execution could otherwise continue after the user has exit (including at the wake up of any sleeping threads). When the user does exit, this is set to false causing all orphan threads to quietly terminate.

3.10 Review

The implementation stage went on the whole, according to plan.

[8]In fact it may be more accurate to say that few parts of the project do *not* use concurrency due to the asynchronous nature of the tasks.

Chapter 4

Evaluation

In this chapter I describe how the success of the project was verified and provide qualitative and quantitative measurements of the performance of key aspects of it.

In addition to the quantitative experiments conducted, the system was trialled multiple times while running or cycling. In all trials it worked as intended, providing accurate (and useful) data.

4.1 Run-Time Performance of Session Setup Procedures

To make a quantitative comparison of the two setup procedures implemented would be difficult because the user experience is hard to quantify. Despite this, the running time of each has been analysed.

In the experiments carried out, to setup a two-device session, both methods took approximately the same time as is shown in Table 4.1. However these measurements are of the minimum setup time, meaning the time for manual

[1]This increased to 16.0 seconds when Wi-Fi was enabled on the device tested, this is explained by the fact that Bluetooth and Wi-Fi share part of the same spectrum and interfere somewhat.[9]

Session Setup Procedure	**Mean Time (s)**
Bluetooth	4.8[1]
QuickStart	4.7

Table 4.1: Timing measurements of the automated parts of both setup procedures, averaged over 20 tests.

selection of participants or the wait before grouping is not considered. In reality, this would take additional time depending on the situation. It is not clear which of the Bluetooth user selection and the server waiting time would take the longest but they are certainly within the same ballpark.

What is significant is that the QuickStart method is highly parallelised since the bottleneck for each device is the network latency to the server, and these bottlenecks are independent, resulting in constant time setup[2]. For the Bluetooth method however, the most time consuming operation is the connection initiation between devices which must be serialised, so the time taken increases linearly with the number of devices.

This makes the QuickStart method faster in all cases[3], but especially so for larger groups.

4.2 Security and Privacy Issues

Since the application involves the transmission of location data along with the names of users, there is potential for breaches of privacy by leaking information to those outside the session, or by false locations being provided to those within.

For the purposes of this evaluation, I will define the term *secure* as ensuring authenticity of the sender of all received messages, and integrity and confidentiality of the content of those messages.

Both session setup procedures provide the ability to distribute cryptographic keys for use in the main communication phase. This presents a multitude of end-to-end encryption options, one of which would be to use a public key system with the following setup[4]:

Each client has their own public/private key pair, and those public keys are distributed with the session at the start. Confidentiality is achieved by encrypting messages with the public key of the recipient, requiring the private key to decrypt them. Authenticity and Integrity are ensured by digitally signing each message with the private key.

The above scheme does not require the use of an existing Public Key Infrastructure (PKI), instead the keys are distributed along with the session object at

[2]Counting the server grouping operations as negligible in comparison with the network latency.

[3]Provided network latency is similar to those at the time of the experiment, and assuming the time taken to manually select participants is equal to the server grouping timeout.

[4]Alternatively if authentication between members of the session is not required, a shared key scheme could be used.

the start. Therefore the security of the entire scheme relies on the security of the session setup procedure.

Security of the Bluetooth Setup Procedure

The physical proximity requirement of Bluetooth transmission aids its security because any attacker would need to be physically present. All transmissions using Bluetooth are encrypted using the key established at the time of pairing, and Fast Frequency-Hopping Spread Spectrum is used for the radio frequency making eavesdropping practically impossible without the key.[10] However it has been shown that with the ability to eavesdrop the pairing process, the key can be feasibly deduced by brute-force, providing access to the later established communication channel.[11][12] For these reasons the Bluetooth method is only *secure* if an attacker is not present at the time of pairing. However since the pairing of Bluetooth devices is an abstraction provided by the OS, and not restricted to, nor within the control of this application, it is reasonable to assume all existent pairings are trustworthy. The application *itself*, is *secure*[5].

Security of the QuickStart Setup Procedure

Simply put the QuickStart procedure is not *secure*. The peer grouping relies solely on location data provided by the client and this data is not validated. An attacker can therefore submit an arbitrary location and be grouped with participants at that location, gaining access to their location for the duration of the session. Thus confidentiality is not provided. In the same way an attacker can spoof their identifier at session setup time, posing as any arbitrary participant. Thus authenticity is also lost. Integrity remains.

This applies to the procedure as implemented, but it is possible to make it *secure* with some, quite significant alterations. One partial fix would be to add a user selection screen after the initial grouping as mentioned in 3.5.2. This would provide confidentiality, since data would only be sent to (and readable by) those selected, but not authenticity as identities are still not verified.

To provide authenticity a PKI could be used with identities being created on installation of the application. When requesting to the server to join a session, the server would respond with a challenge that the client must respond to and sign with its private key. This would authenticate the client to the server, the server would then group the requests as usual and reply with the session object

[5]As always in the field of security, the security of the entire system should be taken into account—it is not sufficient to blindly rely on the application itself.

encrypted to each recipient using their public key. Server authenticity is also required to prevent Man-In-The-Middle (MITM) attacks between the server and client, so all server bound messages would be encrypted using its well-known public key, and it will sign and encrypt all of its outgoing messages.

Now when the user selection occurs, all identities will be unique so selection is no longer ambiguous and authenticity is provided for the duration of the session, as long as all further messages are signed and encrypted using the same keys.

This PKI would be a significant addition to the project but would provide the QuickStart method with the desired *security*.

4.3 Comparison of Network Architectures

To compare the performance of the different protocols, a considerable amount of time was spent working out which information to log, and how to compare the output data.

As described in Section 3.8 the logger outputs a CSV file to disk at the end of each session. Files containing Comma Separated Values (CSV) were chosen as the format because they can be easily imported into spreadsheet software to spot trends. Microsoft Excel was used for manual examination of the graphs produced. A truncated output file is given in Appendix D.

In Excel, a spreadsheet was used to aggregate all the log files from the same session to provide more insightful data such as the transit time between devices.

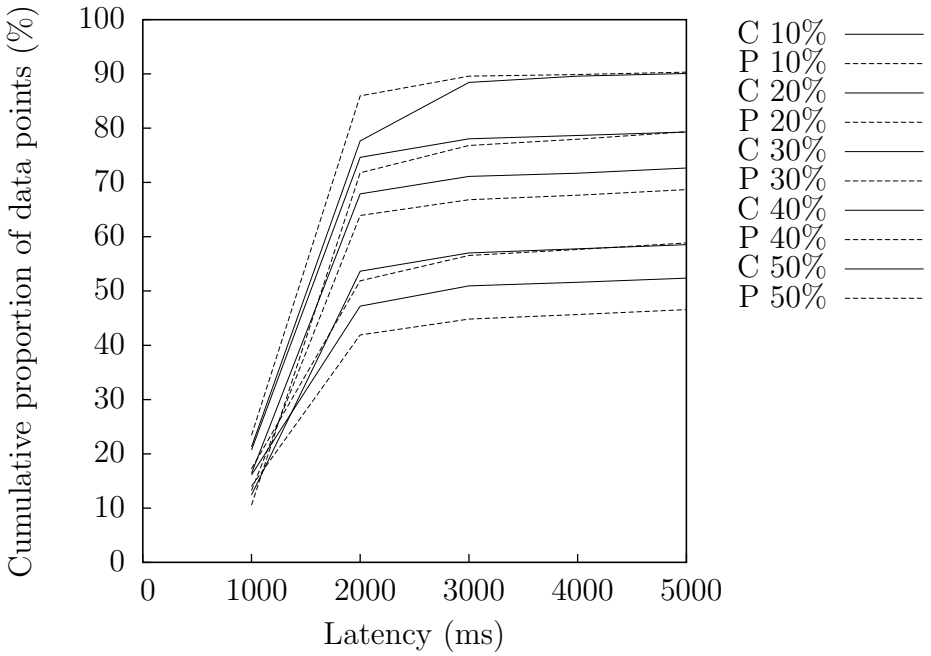


Figure 4.1: Cumulative frequency graph of data point latency. Lines are labelled 'C X' or 'P X' with 'C' representing Client-server, 'P' representing Peer-to-peer and 'X' being the packet drop rate.

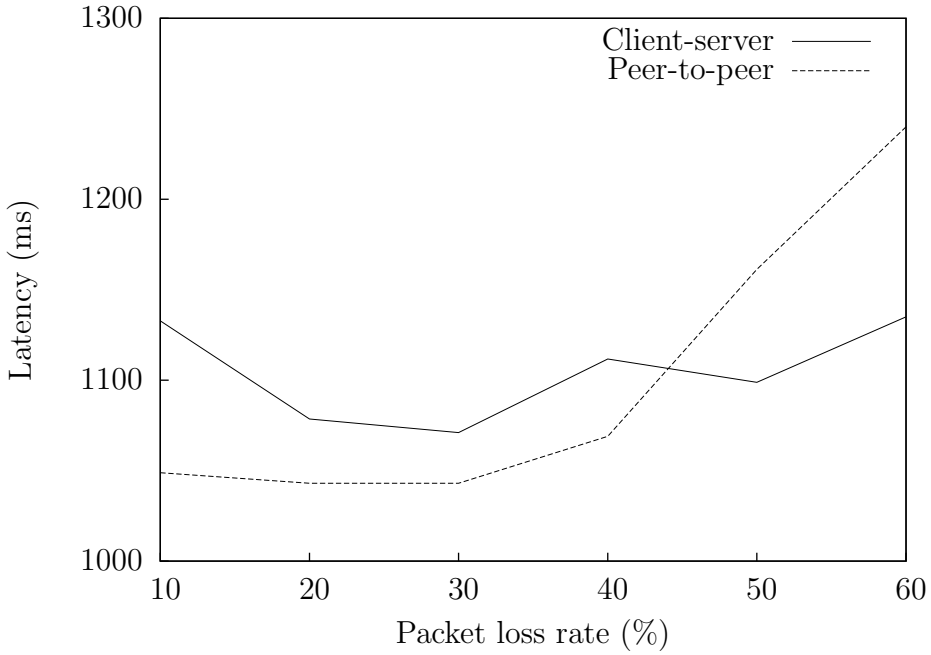


Figure 4.2: Average latency of each topology with increasing packet loss.

To gather the data shown in Figure 4.1, code was added to the client to randomly discard incoming or outgoing packets whenever a dropping mode was enabled. This is to simulate the loss of network packets as expected in the use cases.

Two dropping methods were implemented: an independent Bernoulli distribution, and a two-state Markov chain with states "dropping" and "not dropping"—this was thought to model the real situation more accurately since almost all residual losses[6] occur during temporary disconnection, which likely lasts for some time.[4] For these tests, the system was run on a 3-device session for 10 minutes on each trial. Three trials were done for each 10% increment in drop rate from 10% to 50% and median values were taken.

Figure 4.1 shows that when packet drop rates are low ($< 10\%$), at least 80% of data points are successfully delivered within 2 seconds. This proportion decreases as expected when drop rates increase, with only 40% of deliveries taking less than 2 seconds for drop rates of 50%.

Figure 4.2 shows that for drop rates of $< 40\%$, the average latency of the peer-to-peer system was less than that of the client-server system but increased more quickly with increasing packet loss. Above about 40% the client-server system has the advantage, supporting the theory explained in Section 2.2.3. The initial reduced latency of the peer-to-peer system is likely due to the lack of server processing and routing time.

4.4 How the Application Scales

As planned, the server application was modified to simulate an arbitrary number of devices interacting with a real device running the client application. This allowed scale tests to be conducted with relative ease.

Sustained Throughput, (ST) refers to the successful proportion of all data points averaged over some time frame. Only data after a warm-up period was included, to give the system time to stabilise first. This measures the sustainable throughput of the system, a more insightful measure, for long term use, than the average throughput over the entire duration. Figure 4.4 shows how the average latency and sustained throughput varies for different session sizes.

In maximum load tests the performance benefits of the client-server model were observed. With the server doing all of the coordination and routing, the client applications are spared much computation, this leads to noticeable differences in the scalability of the system.

[6] Residual refers to the losses not overcome by Data-Link (MAC) layer retransmission.

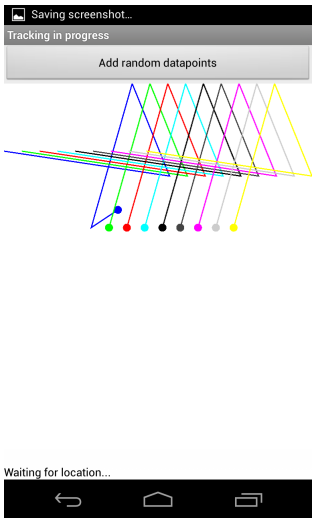


Figure 4.3: Map Display Screen during a simulated 9-device scale test using randomly generated location data.

With each device producing one data point per second the Peer-To-Peer system was rendered useless[7] at 1024 devices per session. At this point, the networking requirements of the application exceeded the device capabilities[8]. This could be improved by sharing data in a more structured arrangement (See Appendix C) so that each device doesn't need to maintain so many connections. In the same conditions, the client-server implementation ceased to be of any use with 2048 devices. Both of these maximum values are far greater than the session size the application was designed for. With session sizes of fewer than 10 devices, the ST is consistently 100%. Nevertheless, with more intelligent grouping of information and more fine-grained locking between threads, it is believed that these limits could be increased if desired.

It is worth noting that in this case, scalability refers to the number of devices in some given session, and not the number of sessions. When the number of sessions is considered, the Bluetooth method will excel in session setup performance, since all setup work is distributed.

4.5 Prioritising Data

Since the distribution of new data is always done in one thread, and requests and re-distribution in another equally prioritised thread, at least half of the CPU

[7]The point at which all incoming updates can no longer be processed in time (ST=0).
[8]Device used for this test was a Google Nexus 1.

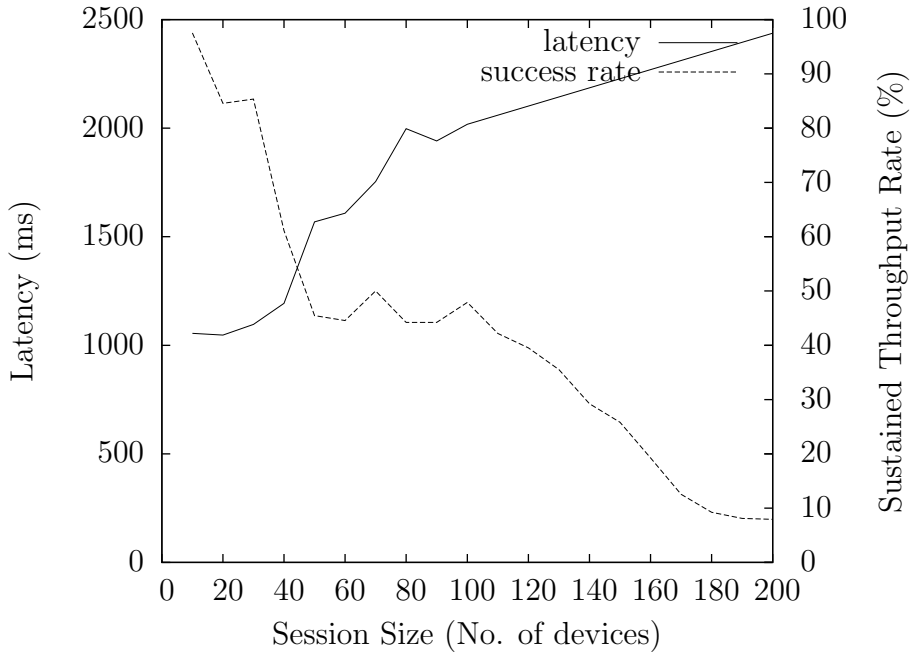


Figure 4.4: Graph showing how average latency and sustained throughput is affected by increasing session sizes when the Peer-to-peer system is in use.

time is always available for processing new data. This prioritising was an original aim in the protocol and has been observed in the results of the tests conducted.

Figure 4.1 shows that the system was able to deliver the majority of data points within 2 seconds whenever the packet drop rate is 40% or less, and in fact Figure 4.2 shows that in every test (up to 60% drop rate) the average latency[9] did not exceed 2 seconds. Also, Figure 4.4 shows that even with an ST of less than 20% the average latency was less than three seconds. This means on average, the points that were drawn ($< 20\%$), were relatively recent ($< 3s$ old), and therefore useful, so the desired prioritising of recent data has been achieved.

4.6 Review

Manual analysis of the logged data to extract meaningful information was more difficult than anticipated, resulting in the evaluation stage taking several weeks longer than expected.

[9]Time between generation of a point and it being drawn to the screen of another device. Note this is *not* the same as network latency.

4.7 Meeting the Original Goals

The initial success criteria outlined in the proposal were:

- App displays an on-screen visualisation of data from the local GPS.
- App plots positions of up to at least four participants (including itself.) in near real-time on the map
- At least one server based method and one peer to peer method of communication has been implemented, with comparisons of their performance made.

All above criteria have been satisfied. Another informal goal in the proposal was to evaluate the security and privacy issues of the system. This was done in Section 4.2.

Chapter 5

Conclusion

As stated in Section 4 all goals have been met and the implementation and evaluation is regarded as successful. During the evaluation I was able to appreciate the difficulties of quantitatively evaluating real life systems. The Iterative development process fitted the project well, and no disasters occurred, though the cellular blocking of peer-to-peer traffic discovered in Section 2.2.4 was a disappointment.

The proposal time-scale turned out to be sensible up to the evaluation stage. If I was to redo the project I would allocate more time for evaluation of the system, and decide on the exact analysis to be done, much earlier.

I am confident that the application is useful in the situations for which it was designed. All practical field tests yielded positive results and the quantitative evaluation shows that the system is theoretically useful.

I plan to continue the development of the system by adding some of the features of Section 5.1 and eventually publishing it.

No conclusive result was found as to the best setup procedure, with QuickStart being the simplest to use, but only the Bluetooth method currently providing the desired security properties.

The evaluation results suggest that for cases of extreme packet loss ($> 40\%$), the client-server topology should be favoured over the peer-to-peer topology, with the opposite being true for less lossy environments. However, the issues described

in Section 3.7 such as server operating costs must also be considered.

5.1 Future Work

There are many ways this work could be extended to provide useful and/or interesting features.

Communication

Currently all communication during the main phase is done through one medium, either Wi-Fi or cellular Internet. There is much scope for the simultaneous use of multiple channels to communicate, mixing infrastructure and ad-hoc methods opportunistically. Methods used in technologies such as Multipath TCP (MPTCP) could also be applied if participants pass through various wireless networks along their course.[13]

A primitive leeching feature was implemented whereby the tethering capabilities of devices are used to share their connection while participating—whenever a non-Internet enabled device is within Wi-Fi range it automatically connects and synchronises its history data. This could be extended to provide extra flexibility.

Different ways of structuring the device space could be used, these are elaborated in Appendix C.

As mentioned in 2.2.4, IPv6 is becoming increasingly adopted, this should eventually eliminate NAT and allow the peer-to-peer implementation built here to operate properly on cellular data networks.

Another desirable feature would be some reflective aspect that alters the protocol parameters based on measurements made. Characteristics of different mobile networks vary from one to another (see Section 2.2.1), as well as in time, so an efficient protocol should make changes according to the current network state.

Application features

A desirable feature would be to have a leaderboard showing the current ranking of participants during the race. It could be augmented with estimates of the time or track distance between users. This would require some method of determining which of two participants is ahead of the other in the race, a decision that may not be trivial when on tight cornering tracks.

The user interface including the track visualiser could also be improved, while still keeping the underlying program core.

Bibliography

[1] *Wi-Fi Direct*,
Wi-Fi Alliance, 2013
http://www.wi-fi.org/discover-and-learn/wi-fi-direct

[2] *Android 2.2 Platform Highlights*,
http://developer.android.com/about/versions/android-2.2-highlights.html

[3] *Android Devices - CPUMark Rating*,
PassMark Software, 2013
http://www.androidbenchmark.net/cpumark_chart.html

[4] *TCP/IP Performance over 3G Wireless Links with Rate and Delay Variation*,
Mun Choon Chan and Ramachandran Ramjee,
Bell Labs, Lucent Technologies, 2013
http://research.microsoft.com/en-us/um/people/ramjee/papers/tcp3g02.pdf

[5] *RFC 2460 - Internet Protocol Version 6 (IPv6)*,
Internet Engineering Task Force, 2013
http://tools.ietf.org/html/rfc2460

[6] *Universal Transverse Mercator System*
Steven Dutch, 2013
http://www.uwgb.edu/dutchs/fieldmethods/utmsystem.htm

[7] *JCoord*
Jonathan Stott, 2013
http://www.jstott.me.uk/jcoord/

[8] *Android API Guides*,
http://developer.android.com/guide/components/index.html

[9] *Wi-Fi and Bluetooth - Interference issues*,
Hewlett Packard, 2013
http://www.hp.com/rnd/library/pdf/WiFi_Bluetooth_coexistance.pdf

[10] *Security of Bluetooth: An overview of Bluetooth Security*
Marjaana Trskbck, 2013
http://www.cs.hut.fi/Opinnot/Tik-86.174/Bluetooth_Security.pdf

[11] *Sniffing the Bluetooth Pairing*
Alberto Moreno Tablado, 2013
http://www.seguridadmobile.com/bluetooth/bluetooth-security/sniffing-the-Bluetooth-pairing.html

[12] *BTCrack Source Code*
Thierry Zoller, 2013
http://blog.zoller.lu/2012/07/btcrack-101-updated-release.html

[13] *Multipath TCP*
Internet Engineering Task Force, 2013
http://datatracker.ietf.org/wg/mptcp/charter/

Appendix A

Project Proposal

John Knox

Socially Enhancing Track Based Sports

Project Proposal

Churchill College

May 16, 2013

Proforma

Name:	**John Knox**
College:	**Churchill College**
CRS-ID:	**JK510**
Project Title:	**Socially Enhancing Track Based Sports**
Project Originator:	Dr John Fawcett
Supervisor:	Dr John Fawcett

Introduction and Description of the Work

This project implements an Android app for use with track based sports.

The app presents a map of the track on screen with the other participants plotted at their corresponding positions, similar to the Heads Up Display on racing video games. The app continuously sends data from each device to each of the others, aiming for sufficient frequency to provide an accurate display at all times. Due to the nature of some track-based sports such as mountain biking, mobile signal strength may be intermittent so the protocol designs will have to expect heavy packet loss. The project is an investigation and evaluation of the effectiveness of different communication protocols in delivering near real-time updates despite the varying and at times adverse network characteristics. Comparison metrics include frequency of position updates, network data usage, and update delay, as well as how the solutions scale with the number of connected devices.

Resources Required

- My Laptop Computer

  Intel Core i3 M380 @ 2.53GHz

  4GB RAM

  Windows 8 running VirtualBox with Linux Mint

- My Android Phone

  ZTE Blade

Starting Point

Part 1B Networking course taken in previous year.

Substance and Structure of the Project

The project will develop the functionality of a basic Android app to visualise the device's knowledge of the current and historic positions of each participant and to record logs for evaluation. The interface created will be a tool for this scientific evaluation; there is no requirement to deliver a polished, production-ready graphical user interface for ordinary users. Nonetheless, this development requires me to become familiar with the Android system, development environment and build/compile tool chain, and time has been allocated for this.

I will design a modular piece of software for the app, leaving a gap for the networking implementation. This will allow me to drop in different implementations of a network module and compare networking approaches and protocols.

Included in the basic visualisation functionality is the capability to draw the track to the screen, and plot and update player positions. The app must use the Android location APIs to turn on the GPS and record its position at regular intervals. The app must drive the various communications devices – WiFi, mobile internet, bluetooth – as required by the network protocol modules.

The motivating use case means there must be a mechanism to define the group of phones that will work together for one track sport session. This project will investigate the networking and security issues in this area and develop a session set-up "screen" for the app. Once the participant group is established, there should be no cross-talk between independent groups and due consideration for data privacy is included. The evaluation will reflect on the security and privacy properties delivered by the session set-up protocol(s).

Once the track sport is underway, further networking issues become interesting. The approaches taken by the alternative communications modules that I will develop will be divided into client-server, and peer-to-peer ("p2p") based methods. The centralised protocols will use a server application built in Java as the coordinator between devices.

The project includes some time to research network protocols, investigating how other applications use the network connectivity and (for example) how Skype tolerates packet loss. The protocols' goals are frequent and up-to-date positions being displayed in near real time. The output of that research will be designs for the network protocols that I will implement and evaluate – at least one client-server protocol and at least one p2p.

The p2p solution will provide the same functionality as client-server but without the need for a server, using mobile internet, wifi, or potentially other channels if possible. However, it is accepted that mobile operators commonly use Network Address Translation and block incoming connections, so in practice a truly p2p protocol operating over mobile internet may not be compatible with most mobile networks in the real world. Despite this, it may provide valuable results to compare against the server based methods and is highly applicable to other networks: private mobile radios, ZigBee, bluetooth etc.

To avoid the need for multiple Android devices, with their inevitable nuances, I will develop a PC client to simulate many devices running the app in order to see how the protocols scale.

I will also spend a large amount of time evaluating the implementations I make, which will involve adding functions to log data and later analysing it to compare the performance of the different approaches.

The project will be composed of the following sections:

- Familiarising myself with the Android framework, libraries, tool chain and development environments.
- Making a basic app and designing and implementing the functionality for displaying location data and for driving the GPS and communications APIs.
- Designing and implementing a secure session set-up procedure.
- Researching network protocols.
- Implementing alternate communication modules, and the server application.
- Testing and evaluation of network modules.
- Writing dissertation.

Possible extensions of the project

The app could be extended to add voice chat support, so that short audio messages could be unicast, multicast or broadcast to the other participants. This extension would place different demands on the network modules and hence motivate the use of different protocol-layer techniques.

A ghost replay function could be added so that users can race against their (or other's) previous performances around the track.

More functionality could be added to the app so that it does further processing on the data, such as producing graphs of speed and other statistics, using local GPS and data from the other participants' records.

A professionally designed user interface could be added and evaluated (using user trials).

Success Criteria

The following should be achieved:

- App displays an on-screen visualisation of data from the local GPS.
- App plots positions of up to at least four participants (including itself) in near real-time on the map.
- At least one server based method and one peer to peer method of communication has been implemented, with comparisons of their performance made.

Timetable

The following is my plan for the project.

"Additional" in this context means that this task is desirable but not necessary so may be dropped if time is short without impacting the core deliverables of the project.

Weeks 1,2 [20th Oct 2012 - 2nd Nov 2012]

Design the modular software structure for the Android app. Draw up initial UML class diagrams and list the required 'screens'.

Learn to use the Android framework, install Android SDK and get a "hello world" app working.

Begin iteratively defining the "hello world" app into the project app. This work continues in the next stage.

Milestone, 2 Nov: UML class diagrams agreed with supervisor; "hello world" app running on an Android phone.

Weeks 3,4 [3rd Nov - 16th Nov 2012]

Finish implementing the basic app from weeks 1,2, including developing code to draw the positions and trails of each participant on the screen.

Develop code to turn on the Android device's GPS and interpret the data to draw trails on the map. Kick-the-tyres test by walking around the college fields and checking the trace resembles the path walked.

Additional: Print real time data and historical statistics (in numeric format) on the device (in numeric format, for later debugging).

Milestone, 16 Nov: Application has all tracking and displaying functionality, with crude testing performed.

Week 5,6 [17th Nov - 30th Nov 2012]

Research and write up possible communication protocols, including how their performance (in the context of this application) can be meaningfully analysed. Write up and meet with supervisor to identify which protocols will be implemented and how their evaluation will work. This research will be used in weeks 7,8,9,10 as well as when writing the final dissertation.

Milestone, 30 Nov: Written up research results submitted to supervisor, and agreement with supervisor on which protocols I will implement.

End of Michaelmas term [30th November 2012]

Week 7 [1st Dec - 7th Dec 2012]

Research and implement a solution for setting up a secure session with other devices. Use knowledge gained from research in weeks 5,6 to decide what information will have to be exchanged to set up the session.

Additional: Implement another session set up method so ease of use, simplicity and security can be compared.

Milestone, 7 Dec: App now complete except for networking module.

Weeks 8,9 [8th Dec - 21st Dec 2012]

Implement the protocols chosen in weeks 5,6 using the session as set up by the work done in week 7.

The implementations will be in the form of modules that will slot into the rest of the application framework interchangeably, so they can be easily compared later on in the project. This will involve making the server application.

This stage will be continued after a 1 week Christmas break.

Week 10 is contingency – the implementation of these protocols is expected to take three weeks, but an extra is included to absorb any slip at this stage in the project, so testing and evaluation can begin on schedule in January.

Milestone, 21 Dec: Now at least one complete implementation of the app.

Christmas week [22nd Dec 2012 - 28th Dec 2012]

No work is scheduled for this holiday period.

Week 11 [29th Dec 2012 - 4th Jan 2013]

Return to finish the work on implementing protocols from the previous stage. Milestone, 4 Jan: App now complete with several different interchangeable communications modules, as well as a server application to serve the android devices.

Weeks 12,13,14 [5th Jan 2012 - 25th Jan 2013]

Carry out full system tests. This is expected to include some unit testing of network protocol state machines and some live functional verification, to be determined (and agreed with supervisor) in light of implementing the code and better understanding where the risks/bugs lie.

Add to the framework so that run-time statistics are logged automatically.

Evaluate the network implementations made in weeks 8,9. Uses knowledge of which parameters/metrics to use from weeks 5,6.

Analyse the runtime logs to produce results that I will use to compare the performance of the network modules.

Prepare progress report for deadline [1st Feb 2013] and presentation for event [7th Feb 2013]

Milestone, 18 Jan: Have informally written up results from the evaluation tests, that can be used to compare the different implementations in the next stage.

Beginning of Lent term [15th Jan 2013]

Week 15,16 [26th Jan 2013 - 8th Feb 2013]

Write introduction chapter and section headings/bullet points for the remaining chapters.

Milestone, 9 Feb: Introduction and section headings with supervisor for review.

Week 17,18 [9th Feb 2013 - 22nd Feb 2013]

Write preparation and implementation chapters.

Milestone, 23 Feb: These chapters submitted to supervisor for review.

Week 19,20 [23rd Feb 2013 - 9th Mar 2013]

Write Evaluation and Conclusion chapters.

Milestone, 9 Mar: These chapters submitted to supervisor for review.

Week 21 [9th Feb 2013 - 16th Mar 2013]

Incorporate review feedback from supervisor.

Milestone, 16 Mar: First complete draft of dissertation submitted to supervisor for review.

End of Lent term [15th March 2013]

Easter vacation [16th March 2013 - 22nd April 2013]

Extensions and additions to the dissertation, as time permits.

Exam revision.

Final deadline: hand in dissertation by noon on 17th May 2013

Version Control and Back-Up

I will be doing all of the main work on my laptop. I will use Git with a remote repository on the PWF system, where updates will be periodically pushed to, as well as Dropbox backing up the project continuously. In the event that my own computer fails I will use the PWF machines to continue.

Appendix B

UML Class Diagram

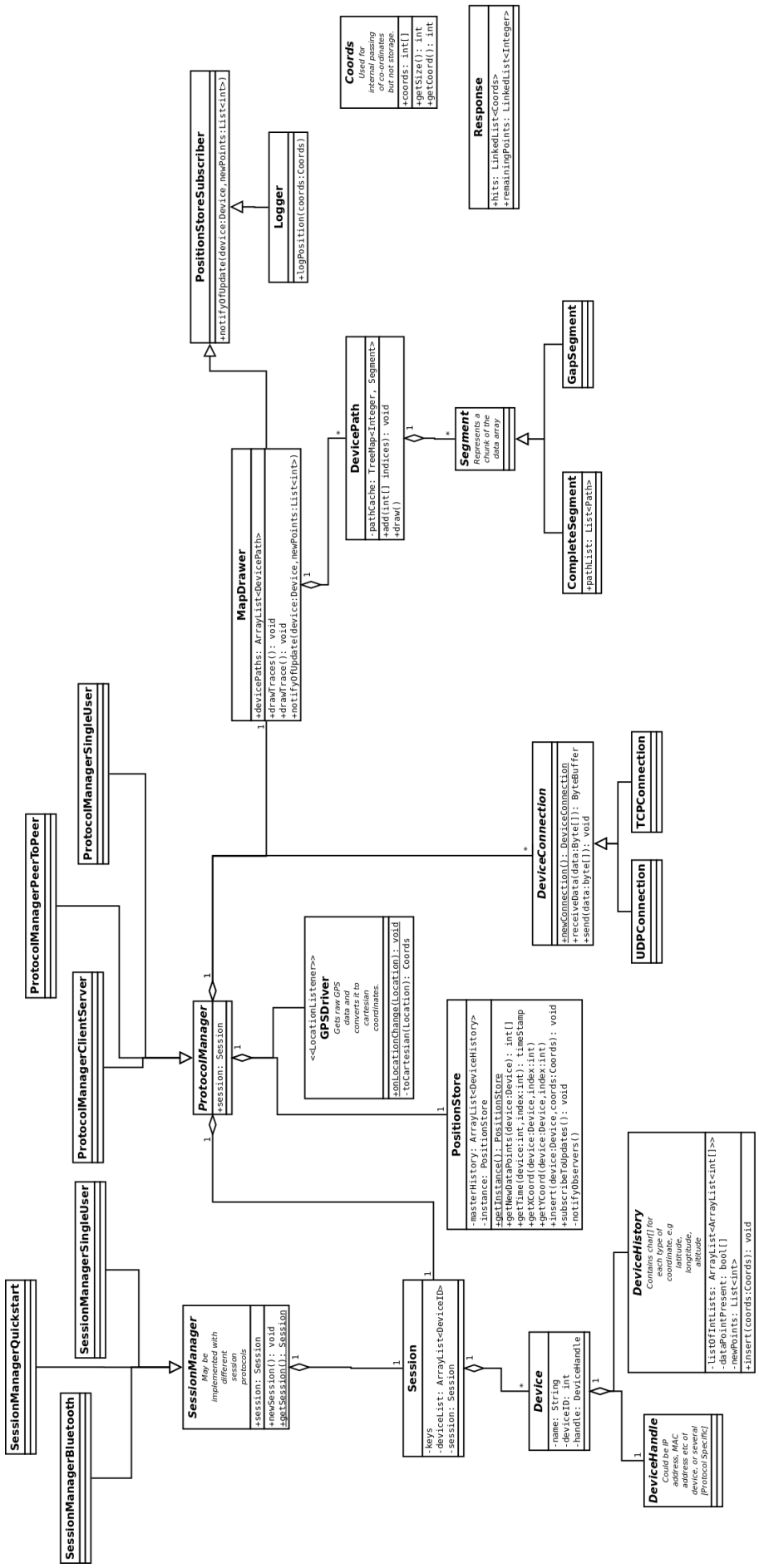


Figure B.1: UML Class Diagram showing the main classes that make up the system.

Appendix C

Structuring the Device Space

In this project, a flat device space is used, meaning that there is no particular association between any two devices.

This does not necessarily have to be the case. When researching possible protocols various different strategies were identified. They have been summarised here to present possible research ideas and future directions for the project.

Given the competitive nature of the application, participants are likely to have more interest in nearby participants than those further away. Short range ad-hoc communication methods would allow for extra throughput between these pairs of participants without putting extra load on the main infrastructural communication with the rest of the session. Technologies identified as potentially useful were Bluetooth and Wi-Fi direct, though there may be more.

Such use of ad-hoc channels would provide much more resilience against loss of network coverage, since clients could distribute their data via other devices that currently have Internet access.

However even if ad-hoc communication is not used, it may be the case that structuring the device space could still prove advantageous. One advantage could be scalability, since devices only have to maintain connections with their neighbours. If we assume, as above that data concerning nearby devices is more important than data about devices further away, one could group devices by location and prioritise sending from each device to members of its own group.

If cellular signal strength is particularly variable among the different devices and their networks, then it may be worthwhile electing a leader of each group with the leaders being those devices with the strongest signal strength / lowest loss rate, and having potentially all inter-group communication go via the elected leaders.

Other ways of grouping devices are mutual or absolute loss rate. If grouping by mutual loss rate, devices would be paired with at least two others with whom

they share a strong mutual connection, in order to make better use of the strong links rather than the weaker ones. If each device paired with its two closest links, with some forceful guidance a ring topology could be formed for example. This would probably be less applicable when using cellular Internet since mutual loss rates do not vary much (though latency may) across different network providers.

To group by absolute loss rate would be to organise the groups so that each contains an approximately uniform distribution of signal strengths among devices. The reason would be to attempt to give all "weaker" devices roughly equal exposure to "stronger" devices that could carry some of their load.

Possible methods for estimating the signal strength of devices would be to dynamically analyse statistics on the go, or use some "incompleteness" metric such as the percentage of data that each device is missing.

If devices were grouped with leaders elected in each, and all inter-group communications going via the leaders, this could be extended to form a hierarchical tree structure, which may have benefits in some conditions.

Appendix D

Example Log File

Topology	clientServer					
Transport	UDP					
Responce heuristic	always					
Packet Dropping	TRUE					
Markov dropping	FALSE					
Connection lose rate	0.08					
Reconnect rate	0.4					
Packet drop rate (if not markov)	0.1					
Devices	3					
This device:	0					
Sent	8428					
Received	80640					
Requests sent	19					
Requests received	17					
Requests responded to	17					
Packets dropped	120					
Generation time						
833	1835	2837	3840	4844	5847	...
Device 0						
834	1835	2837	3840	4844	5848	...
Device 1						
1409	2432	3457	4384	5399	6430	...
Device 2						
2022	3046	4078	5097	6123	7138	...
Request sizes						
0	0	0	0	0	0	...

Table D.1: A truncated .csv file output by the logger function at the end of a session, before aggregation.

Appendix E

Format of Messages

```
enum MessageType {
PayloadMessage = 0;
RequestMessage = 1;
}

message DatapointSet {
repeated Datapoint datapoint = 1;
}

message Datapoint {
required int32 aboutDeviceID = 1;
required int32 logicalTime = 2;
required float xCoord = 3;
required float yCoord = 4;
required float altitude = 5;
}

message RequestSet {
repeated Request request = 1;
}

message Request {
repeated int32 index = 1;
}

message Packet {
required MessageType type = 1;
required int32 fromDeviceID = 2;
optional int32 sessionID = 3;
optional DatapointSet datapoints = 4;
optional RequestSet requests = 5;
}
```

Figure E.1: The format of the Payload and Request Messages sent between devices, described using Protocol Buffers, a language-neutral protocol/format description language and serialization tool. The "Packet" message represents the main packet format, it is built up from the sub-messages defined above it.

Appendix F

PathCache Source Code

Source code for inserting into the PathCache structure:

```
//add new point to this DevicePath
protected synchronized void add(int index, float x, float y) {

        //      get target GapSegment
        Entry<Integer, Segment> gapEntry = pathCache.floorEntry(index);
        GapSegment gap = (GapSegment) gapEntry.getValue();

        /*      if new point is at start of a gap:
                add it the end of the previous segment
                and increment the position of this one.
        */
        if(index == gapEntry.getKey()) {
                //move start of gap one space forward
                pathCache.remove(index);
                //if its not at the end of its gap, add a new gap after it
                if(!pathCache.containsKey(index+1)) {
                        pathCache.put(index+1, gap);
                }
                //get previous segment
                Entry<Integer, Segment> prevEntry =
                        pathCache.lowerEntry(index);

                //if there is a previous segment, add this point to it.
                if(prevEntry != null) {
                        CompleteSegment prev =
                                (CompleteSegment) prevEntry.getValue();
                        Path path = prev.path;
                        path.lineTo(x, y);
                } else {
                        /*otherwise: this is the very first data point,
```

```
                give it it's own path, starting from x, y.
                */
                assert(index == 0);
                Path path = new Path();
                path.moveTo(x, y);
                CompleteSegment comp =
                        new CompleteSegment(x, y, path);
                pathCache.put(index, comp);
        }
    }
    /*else split the gap into a first half gap,
     single point segment, and last half gap.
    */
    else {
        //first half gap
        pathCache.put(gapEntry.getKey(), gap);
        //make single item path containing this point
        Path newPath = new Path();
        newPath.moveTo(x, y);
        CompleteSegment seg =
                new CompleteSegment(x, y, newPath);
        pathCache.put(index, seg);
        /*check for null,
        if so then this is the highest segment yet.
        */
        if(pathCache.higherKey(index)==null) {
                pathCache.put(index+1, gap);
        } else {
                //if theres space after it, add a gap
                if(pathCache.higherKey(index)!=index+1) {
                        pathCache.put(index+1, gap);
                }
        }
    }
    if(index>lastIndex) {
        lastIndex = index;
        endX = x;
        endY = y;
    }
}
```